

Class 9 Hashing, Salting, & Encryption For SOC

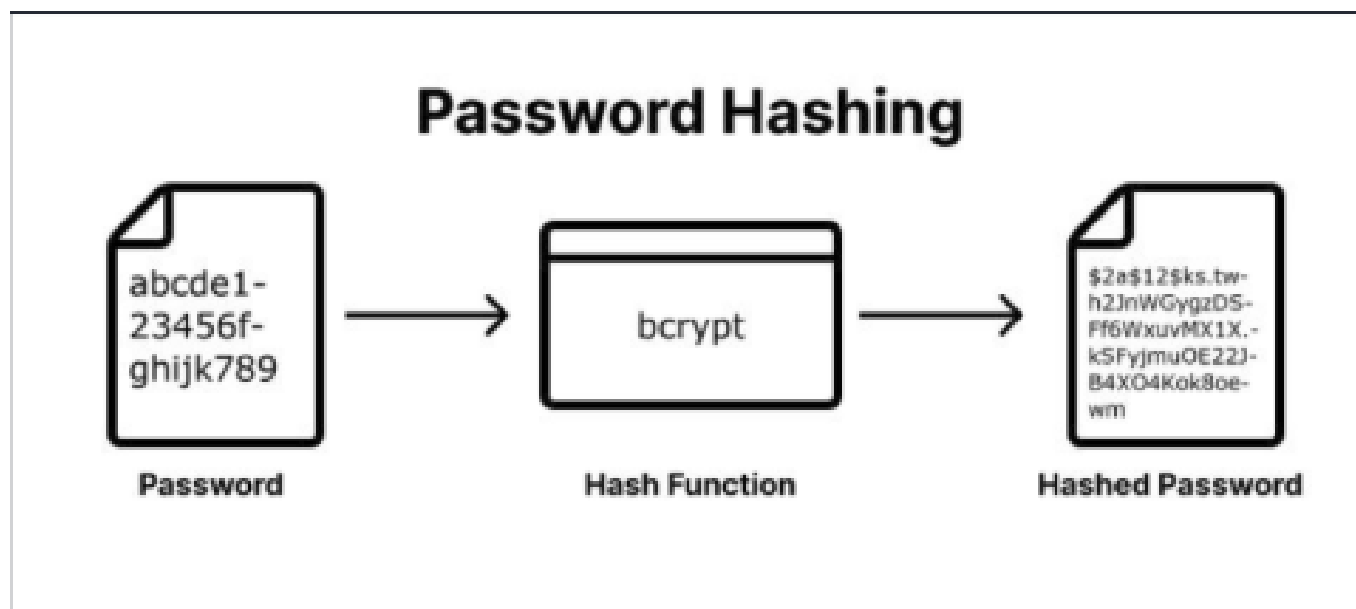
What is Hashing?

A **hash** is a one-way process that converts any data into a **fixed-length string** of characters.

Example:

- Input: `password123`
- MD5 Hash: `482c811da5d5b4bc6d497ffa98491e38`

Key point: No matter if the input is short or long, the hash output always has the **same fixed length** for a given hash algorithm. It is called **one-way** because you cannot easily recover the original data from the hash.



Properties of Hashing

- **One-way:** You cannot easily convert the hash back to the original data.
- **Deterministic:** The same input always produces the same hash.
- **Fixed output length:** The hash is always the same length, regardless of input size.
- **Avalanche Effect:** A tiny change in the input creates a completely different hash.

Example:

- password123 → Hash A
- password124 → Hash B (completely different)

Key point: Even changing one character results in a totally different hash. This is called the **Avalanche Effect**.

Common Hash Algorithms

Algorithm	Secure Today?	Notes
MD5	no	Very Weak
SHA1	no	Broken
SHA256	yes	Strong
SHA512	yes	Strong
bcrypt	Very strong	Designed for passwords
Argon2	Very strong	Modern Standard

Why Hash Passwords?

Passwords are **hashed** to keep them secure.

Without hashing:

- Username: nitesh
- Password: Nitesh@123
- If the database is hacked, attackers can see the real password.

With hashing:

- Username: nitesh
- Hash: 5e884898da...
- If the database is hacked, attackers only see the hash, **not the actual password**.

Key point: Attackers must **crack the hash** to find the original password, making it much harder to steal passwords.

The Big Problem (Without Solving)

Hashing **alone is not enough** for password security.

- If **two users have the same password**, they will get the **same hash**.
- An attacker can easily tell they are using the same password.

Example:

- User 1: password123 → Same hash
- User 2: password123 → Same hash

Rainbow Table Attack

A **rainbow table** is a precomputed list of common passwords and their hashes.

Example:

Password	Hash
123456	e10adc3949ba...
password	5f4dcc3b5aa...
admin123	0192023a7bb...

If a stolen hash matches one in the table, the attacker can **instantly find the password**.

Key point: Older hash algorithms like **MD5** and **SHA-1** are insecure for storing passwords because they are vulnerable to rainbow table attacks.

What is Salting?

A **salt** is a **random value** added to a password **before hashing**.

Instead of hashing:

- password123

The system hashes:

- password123 + random salt

Example:

- Password: password123
- Salt: x7K9pL2

- Combined: password123x7K9pL2
- Hash Output: a83bd981fa...

Key point: Even if two users have the **same password**, each gets a **different random salt**, so their hashes are **different**. This makes rainbow table attacks much harder.

Why Salting is powerful?

Even if **100 users** use the **same password**:

- **Without salt:** All users have the **same hash**.
- **With salt:** Every user gets a **different random salt**, so each hash is **different**.

Key point: Attackers **cannot use rainbow tables effectively**. They must **brute-force each hash separately**, making password cracking much slower and more expensive.

How Salting is Stored (Short Explanation):

The database stores:

- **Username**
- **Salt**
- **Hash**

When a user logs in:

1. The system retrieves the user's **salt**.
2. It adds the salt to the entered password.
3. It hashes the combined value.
4. It compares the new hash with the stored hash.

If both hashes match → Login is successful.

Hashing vs Encryption (Very Important Difference)

Feature	Hashing	Encryption
Reversible ?	No	Yes
Used for password ?	Yes	No
Needs Key ?	No	Yes
Purpose	Integrity	Confidentiality

SOC Perspective: Why You Must Understand This (Short Explanation)

During an **incident response**, if a password database is leaked, a SOC analyst should check:

- Was **password hashing** used?
- Was **salting** used?
- Which **hashing algorithm** was used?
- Was it **bcrypt** (secure) or **MD5** (weak)?

Key point: If weak hashing (like MD5 without salt) was used, there is a **high risk of credential stuffing attacks**, where attackers try the cracked passwords on other websites.

Real Attack Scenario (Short Explanation)

1. A company stores passwords using **MD5 without salt**.
2. The database is hacked.
3. Attackers **crack the hashes quickly**.
4. Users have reused the same password for:
 - Email
 - VPN
 - Banking
5. Attackers launch a **credential stuffing attack** using those passwords.

What the SOC team sees:

- Many login attempts from different IP addresses.
- User account takeovers.

- Impossible travel alerts (logins from distant locations in a short time).

Root Cause: Weak password hashing practices (using MD5 without salting).

Advanced Concept: Pepper

A **pepper** is a **secret value** stored **outside the database** (for example, in the server's configuration or a secure secret manager).

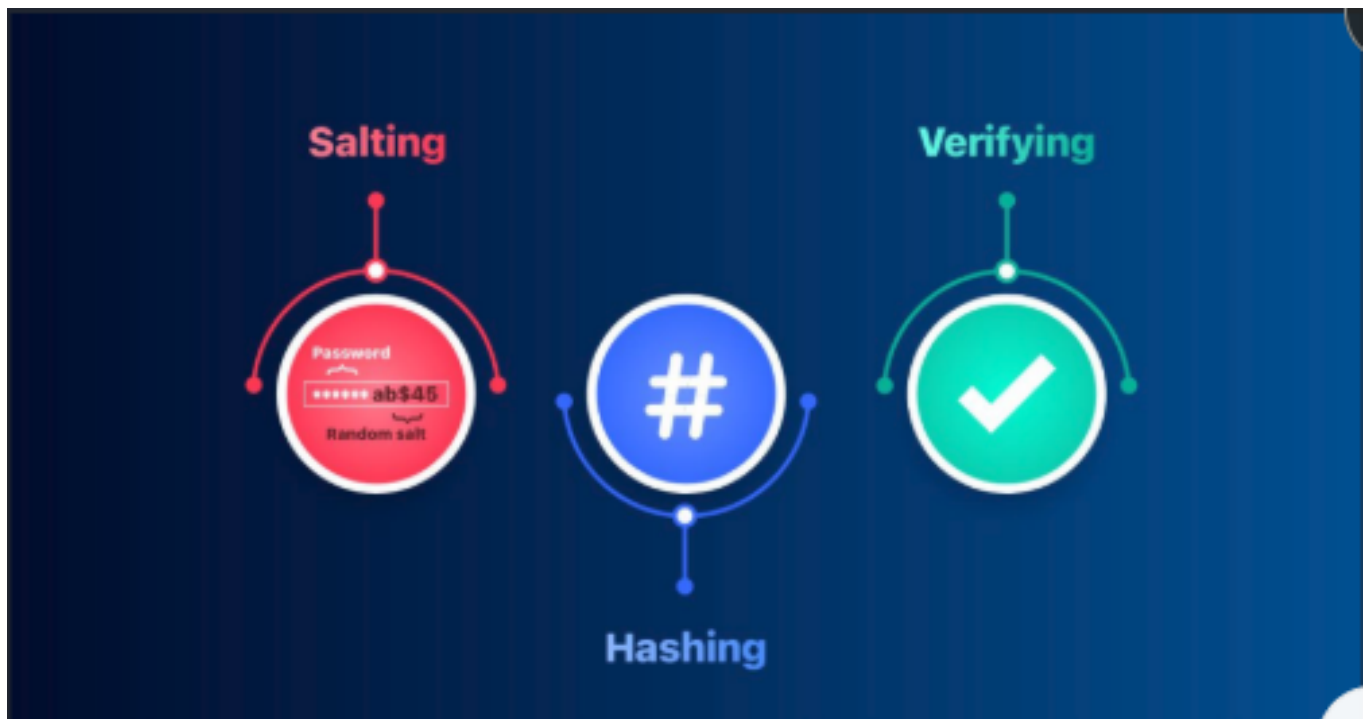
Instead of hashing:

- password + salt

The system hashes:

- password + salt + pepper

Key point: Even if the database is leaked, the attacker **does not know the pepper** because it is stored separately. This provides an **extra layer of protection** and makes it much harder to crack passwords.



What is Encryption?

Encryption is the process of converting **readable data (plaintext)** into **unreadable data (ciphertext)** using a **cryptographic key**. Only someone with the **correct key** can decrypt it back to the original data.

Basic Flow:

- Plaintext → Encryption + Key → Ciphertext
- Ciphertext → Decryption + Key → Plaintext

Key point: Unlike hashing, **encryption is reversible** if you have the correct key.

Example:

- Original message: BankTransfer = ₹50,000
- Encrypted message: A7F9C1B2E88DA91...

With the correct key, the encrypted message can be decrypted back to:

- BankTransfer = ₹50,000

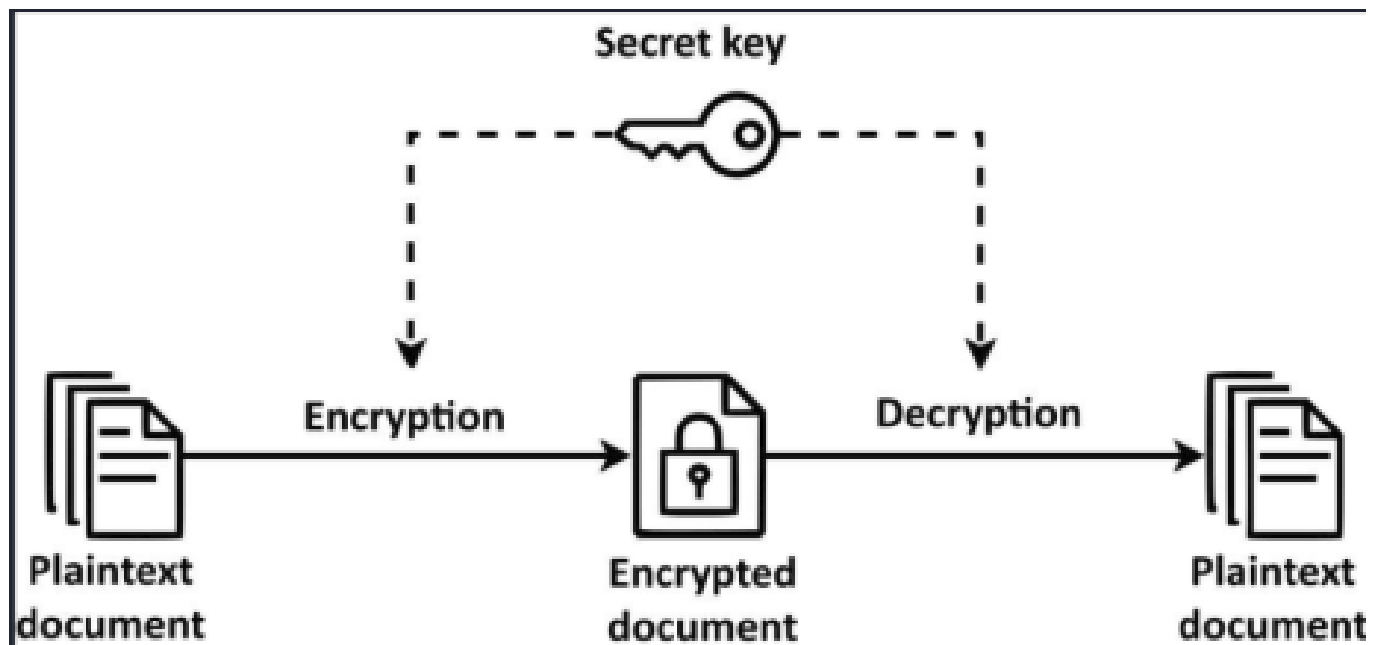
Why Encryption is Used

Encryption is used to protect **data confidentiality**, meaning **only authorized users with the correct key can read the data**.

Common uses of encryption:

- **HTTPS** website traffic (secure browsing)
- **VPN** communication
- **Disk** encryption
- **Secure messaging** apps
- **File** encryption
- **Database** encryption
- **Email** encryption

Key point: Encryption keeps sensitive data safe from unauthorized access during storage or transmission.



Symmetric Encryption

Symmetric encryption uses **one secret key** for both **encryption** and **decryption**.

Basic Flow:

- **Plaintext + Secret Key → Ciphertext**
- **Ciphertext + Secret Key → Plaintext**

Popular Symmetric Encryption Algorithms

- **AES** – Most common and secure modern encryption.
- **DES** – Old and insecure.
- **3DES** – Legacy algorithm, largely replaced by AES.
- **ChaCha20** – Modern, fast, and secure encryption.

Common Uses

- Disk encryption
- VPN tunnels
- File encryption

Key point: The biggest challenge with symmetric encryption is **securely sharing the secret key** between the sender and receiver. If the key is stolen, anyone with it can decrypt the data.

Asymmetric Encryption

Asymmetric encryption uses **two different keys**:

- **Public Key** – Shared with everyone and used to **encrypt** data.
- **Private Key** – Kept secret and used to **decrypt** data.

Basic Flow:

- **Public Key** → **Encrypt**
- **Private Key** → **Decrypt**

Popular Algorithms

- **RSA** – Commonly used in SSL/TLS.
- **ECC (Elliptic Curve Cryptography)** – Modern, secure, and efficient.
- **Diffie-Hellman** – Used for **key exchange** (not for encrypting data directly).

Common Uses

- HTTPS websites
- SSL/TLS handshake
- Email encryption (PGP)

Key point: Anyone can use the **public key** to encrypt data, but **only the owner of the private key can decrypt it**, making it secure for communication over the internet.

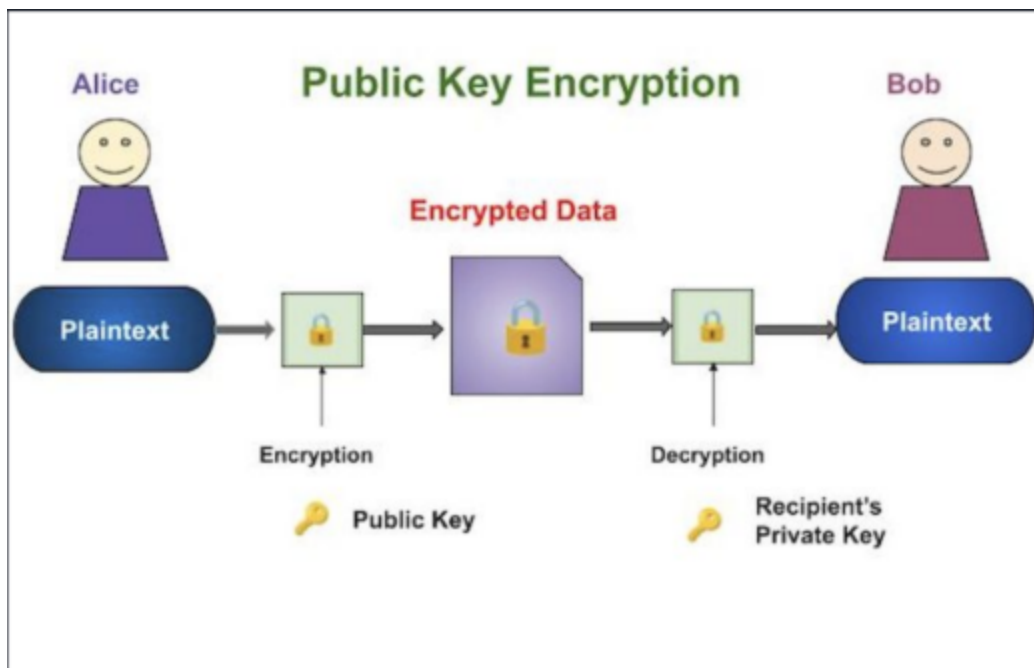
HTTPS Example: Encryption

When you visit an **HTTPS website**, your browser and the server create a secure connection.

Steps:

1. The **browser gets the server's public key**.
2. The **browser encrypts a session key** using the public key.
3. The **server decrypts the session key** using its **private key**.
4. Both the browser and server use the session key for **secure, encrypted communication**.

Key point: HTTPS uses **asymmetric encryption** (public/private keys) to securely exchange a **session key**, then uses **symmetric encryption** (such as AES) for fast, secure communication.



Encryption vs Hashing

feature	Encryption	Hashing
Reversible	YES	NO
Uses key	YES	NO
Purpose	Protect data confidentiality	Verify data integrity
Output Length	Variable	Fixed
Example use	HTTPS traffic	Password storage
Can original data be retrieved	YES	NO

SOC Analyst Perspective (Short Explanation)

Understanding **hashing** and **encryption** helps a SOC analyst with:

- Investigating **data breaches**
- Analyzing **password dumps**
- Understanding **credential theft**
- Investigating **encrypted network traffic**
- Performing **malware analysis**

Example SOC Scenario

If attackers steal:

- **An encrypted database** → The data is **hard to read** without the decryption key.
- **Hashed passwords** → Attackers may try to **crack the hashes offline** using brute-force or dictionary attacks.

Key point: As a SOC analyst, you should determine whether the stolen data was **encrypted** or **hashed**, because this affects how serious the breach is and how attackers might exploit the data.